Let’s now reason about which one, among the basic data structures we have seen so far, could guarantee us the best performance. Well, handling the size of the cache is easy, we can keep a variable for that and update it in constant time, so we won’t mention it any further in this analysis.

Checking if an entry exists is based on finding an entry by name, so only the latter operation needs to be analyzed.

Storing and retrieving entries by name should ring a bell: it’s clearly what an associative array does, so hash tables would be the obvious choice there. Hash tables, however, are not great when it comes to retrieving the minimum (or maximum) element they contain[18].

In fact, removing the oldest element could take up to linear time and, unless the expected life of the cache is so short that, on average, we are not going to fill it completely, this could slow down every insertion of a new entry.

Using arrays doesn’t seem a good idea, as they would not speed up any of the operations, or at most just a single one, while requiring all the space for maximum capacity to be allocated from the start.

Linked lists seem promising for keeping the order of insertion, but they wouldn’t be great when it comes to looking up entries.

If you have read appendix C, you might remember that there is a data structure offering a compromise between all of these different operations: balanced trees! With a tree, we